

Apply filters to SQL queries

Project description

As a security professional, I used SQL queries to investigate potential security incidents and identify employee devices that required security updates. By applying SQL filters, I retrieved specific records from the **log_in_attempts** and **employees** tables based on dates, times, locations, and departments. These queries supported security investigations and helped ensure the appropriate employee machines received the required updates.

Retrieve after hours failed login attempts

SQL Query

```
SELECT *  
FROM log_in_attempts  
WHERE login_time > '18:00'  
AND success = FALSE;
```

Explanation

This query retrieves all failed login attempts that occurred after **18:00 (6:00 PM)**. I selected all records from the **log_in_attempts** table and used a **WHERE** clause with the **AND** operator to apply two conditions. The first condition, `login_time > '18:00'`, filters login attempts that occurred after business hours. The second condition, `success = FALSE`, filters only unsuccessful login attempts. Using **AND** ensures that both conditions must be true for a record to be returned.

Retrieve login attempts on specific dates

SQL Query

```
SELECT *  
FROM log_in_attempts  
WHERE login_date = '2022-05-09'
```

```
OR login_date = '2022-05-08';
```

Explanation

This query returns all login attempts that occurred on **May 9, 2022**, or **May 8, 2022**. I selected all records from the **log_in_attempts** table and used a **WHERE** clause with the **OR** operator. The first condition filters login attempts that occurred on May 9, while the second filters attempts from May 8. Using **OR** allows the query to return records that match either date, making it useful for investigating activity surrounding the suspicious event.

Retrieve login attempts outside of Mexico

SQL Query

```
SELECT *  
FROM log_in_attempts  
WHERE NOT country LIKE 'MEX%';
```

Explanation

This query identifies all login attempts that occurred outside of Mexico. I selected all records from the **log_in_attempts** table and used **NOT** with the **LIKE** keyword. The pattern **'MEX%'** matches both **MEX** and **MEXICO** because the percentage sign (%) is a wildcard that represents any number of additional characters following "MEX". Using **NOT LIKE** excludes all records from Mexico and returns only login attempts originating from other countries.

Retrieve employees in Marketing

SQL Query

```
SELECT *  
FROM employees  
WHERE department = 'Marketing'  
AND office LIKE 'East%';
```

Explanation

This query retrieves all employees in the **Marketing** department who work in the **East** building. The condition `department = 'Marketing'` filters employees by department, while `office LIKE 'East%'` filters office locations that begin with "East", such as **East-170** or **East-320**. The **LIKE** keyword is used to search for a pattern, and the `%` wildcard matches any remaining characters. The **AND** operator ensures both conditions are satisfied before a record is returned.

Retrieve employees in Finance or Sales

SQL Query

```
SELECT *  
FROM employees  
WHERE department = 'Finance'  
   OR department = 'Sales';
```

Explanation

This query retrieves all employees who work in either the **Finance** or **Sales** departments. I selected all records from the **employees** table and used a **WHERE** clause with the **OR** operator. The first condition filters employees in Finance, and the second filters employees in Sales. Using **OR** returns records that belong to either department.

Retrieve all employees not in IT

SQL Query

```
SELECT *  
FROM employees  
WHERE NOT department = 'Information Technology';
```

Explanation

This query identifies all employees who are **not** in the **Information Technology** department. I selected all records from the **employees** table and used the **NOT** operator to exclude employees whose department is Information Technology. This returns employees from every other department so the remaining security update can be applied.

Summary

In this project, I used SQL filters to investigate potential security incidents and identify employee devices that required security updates. I queried the **log_in_attempts** and **employees** tables using **WHERE** clauses with **AND**, **OR**, and **NOT** operators to filter data based on dates, times, departments, and locations. I also used the **LIKE** keyword with the % wildcard to search for patterns, enabling me to efficiently retrieve the information needed to support security investigations and update employee systems.